MOBILE DEVELOPMENT

MIDTERM LABORATORY EXAMINATION

Dart Programming Fundamentals • Object-Oriented Programming

Name: Laroga, Joshua A. **Section:** 3.8BSIT **Date:** 11/10/26

Score: _____ / 50

EXAM OVERVIEW

Total: 50 points **Coverage:** Weeks 7–10 (CO3) **Output:** One integrated Dart console application **Scenario.** Develop a console-based Community Learning Center Management System. The application will register learning resources and participants, process simple transactions, and display organized records. Your program must demonstrate fundamental Dart syntax and object-oriented design in one working application.

I. General Instructions

- Create one Dart console application using DartPad (Console) or the Dart environment specified by your instructor.
- Your program must be original, executable, and organized. Use meaningful class, variable, and method names.
- The application must accept user input, validate choices where appropriate, process records, and display readable output.
- Use Lists and Maps to store and organize records. Do not replace the required OOP structure with a collection of unrelated functions.
- Save your source code as main.dart. Submit the source code and a short README.md containing your name, section, application features, and run instructions in one GitHub repository. • Before submission, test the program using at least three different input scenarios, including one invalid input or error condition.

II. Required Application Features

Implement all requirements below in the same application. The point values indicate the assessed components.

A. Program Setup, Variables, Input/Output, and Operators — 8 points

- Display an application title and a menu with at least three actions (for example: add resource,

register participant, view records, exit).

• Use appropriate variables and Dart data types (String, int, double, bool, or inferred types). • Accept and process user input using console input/output. Use operators for at least one meaningful calculation or comparison.

B. Conditional Statements, Loops, Functions, and Error Handling — 12 points • Use if/else or switch to process menu selections and application rules.

- Use an appropriate loop to keep the menu running until the user chooses to exit, and/or to display multiple records.
- Create at least two user-defined functions with parameters and meaningful return values or actions.
- Handle invalid menu choices and invalid or missing input without allowing the application to terminate unexpectedly.

C. Classes, Objects, Constructors, Methods, and Encapsulation — 10 points

- Create at least two related classes (e.g., LearningResource and Participant) with appropriate fields, constructors, and methods.
- Instantiate and use objects in the application.
- Demonstrate encapsulation using private fields (underscore prefix) and controlled access through methods or getters/setters.
- Use a List and a Map meaningfully to store or retrieve application records.

D. Inheritance, Polymorphism, Abstract Class/Interface, and Mixins — 12 points • Create an abstract parent class or interface representing a general center record or user. • Create at least two subclasses that inherit from the parent and provide their own implementation of a shared method.

- Demonstrate polymorphism by handling different subclass objects through a shared parent type or method.
- Use at least one mixin to provide reusable behavior (e.g., timestamping, printable details, or status tracking).

E. Integration, Testing, and GitHub Submission — 8 points

- Ensure all required components work together as one application and the menu flow is understandable.
- Test at least three scenarios and show that invalid input is handled appropriately.

- Submit main.dart and README.md in one accessible GitHub repository. The repository must contain the final working version.

III. Minimum Technical Expectations

- The application must compile and run. A program that does not execute will receive only the points supported by the submitted code and demonstrated evidence.
- Use clear prompts and readable output. Avoid hard-coded sample output in place of actual program logic.
- Use try-catch or equivalent validation where appropriate. Explain or display a useful message when an operation cannot be completed.
- Your class hierarchy must be meaningful. Do not add inheritance, mixins, or abstract classes solely to satisfy a keyword requirement.
- Do not submit multiple unrelated programs. All requirements must be integrated into one application.

IV. Submission Requirements

- GitHub repository name: MD_Midterm_[Surname]_[Section]
- Required files: main.dart and README.md.
- README.md must include: application title, short description, implemented features, sample test scenarios, and instructions for running the program.
- Set repository visibility to Public or provide instructor access. Verify the repository link before submitting it through the designated submission channel.
- Submit the repository link only once. Ensure that the latest commit contains the final version of your work.

V. Scoring Rubric

Program setup, variables, I/O, operators	8	Complete and accurate use of Dart fundamentals; input/output and operations work correctly.	Mostly correct; minor omissions or errors.	Limited or partially working implementation. Missing or nonfunctional.
Control structures,	12	Correct logic, effective loops/functions, and	Most logic works; minor errors or	Several logic errors; limited function or error

functions, error handling		graceful handling of invalid input.	incomplete handling.	handling. Major requirements absent or nonfunctional.
OOP: classes, constructors, methods, encapsulation, collections	10	Well-designed classes and objects; correct encapsulation; Lists/Maps used meaningfully.	Sound structure with minor design or implementation issues.	Basic or incomplete OOP; weak encapsulation/collection use. Required OOP elements missing.
Advanced OOP: inheritance, polymorphism, abstraction, mixin	12	Meaningful hierarchy; correct overriding/polymorphism; appropriate abstraction and mixin.	Most concepts demonstrated with minor issues.	Concepts attempted but poorly integrated or partly incorrect. Most advanced OOP requirements absent.

Integration, testing, GitHub submission	8	Integrated, tested, readable application; complete and accessible repository.	Application mostly integrated; minor testing or documentation gaps.	Partial integration; limited testing or incomplete repository. No verifiable working submission.

Scoring note: The instructor will assign a score within each criterion based on the quality, completeness, correctness, and demonstrated functionality of the submitted application. Total: 50 points.

VI. AI Use Policy

- AI tools may be used for brainstorming, clarifying Dart concepts, improving comments or documentation, and understanding or debugging a specific error.
- AI assistance is limited to a maximum of 20% of the submitted work. The core program design, class structure, logic, implementation, and testing must be the student's own work. • Do not submit an AI-generated complete solution or copy code that you cannot explain. Fabricated authorship or undisclosed substantial AI-generated code may be treated as an academic integrity violation.
- Include a short AI Use Declaration in README.md stating whether AI was used, the tool (if any), the specific purpose, and the part of the work assisted. If no AI was used, write: "No AI tools were used."
- The instructor may ask you to explain, modify, or demonstrate any part of your code. Inability to explain submitted work may affect the score and may be referred for academic integrity review.

VII. Deduction Policy

Non-executable or substantially broken program	Up to 10 points may be deducted from the affected criteria, depending on the extent of the failure. Missing or nonfunctional features receive no points under the relevant criterion.
Missing required file or incomplete GitHub repository	Up to 3 points deducted from Integration, Testing, and GitHub Submission.
Unreadable, disorganized, or unexplained code	Up to 2 points deducted from the affected

	implementation criterion.
Hard-coded output in place of required logic / unrelated or duplicated code	No credit for the affected requirement; additional deductions may apply within the relevant criterion.
Late submission	A 5-point deduction from the total score applies to late submissions, subject to the instructor's announced submission deadline and class policy. No extension is automatic.
Academic dishonesty or misrepresented authorship	May result in a zero for the examination and referral in accordance with institutional academic integrity policies.

Important: Deductions are applied within the stated rules and must not reduce the examination score below zero. A single issue will not be double-penalized under multiple categories unless it represents separate violations.

VIII. Student Declaration

I certify that the submitted application is my own work, that any permitted assistance has been properly declared, and that I can explain and demonstrate the code I submitted.



Student Signature: _____ Date: _____